

Tagueur intégré à HyperBase

Philippe
septembre 2021 – septembre 2022

Le format APN

Je n'ai jamais eu de description complète et officielle du format APN. C'est un format fixe avec 80 caractères par ligne : chaque ligne correspond à un lemme. Outre la forme, le lemme et l'analyse, une ligne contient des informations comme le code de l'œuvre, le numéro de la phrase et un moyen de référencement. Le tout est organisé sous forme de champs de longueur fixe. J'ai eu en décembre 2015 une réponse de Laurent Simon sur le nombre de caractères par champ :

- 3 car. pour le code d'oeuvre,
- le car. n° 4 (& : par défaut, = : enclitique, # : une forme pour plusieurs lemmes -> formes type 'abductast' ou nombres, + : je ne sais plus, il n'y en a que 6 chez Plinie, probablement pas très important –dixit Laurent),
- 4 car. pour le numéro de phrase,
- 21 car. pour le lemme,
- 1 car. pour l'indice d'homonymie,
- 25 pour la forme,
- 11 pour la référence,
- 1 car. vide,
- 9 pour l'analyse,
- 2 pour les codes de subordination

(on est à 78 car.), les 2 derniers sont probablement vides avec le n° 80 qui est le passage à la ligne, en tout cas, je ne les utilise pas, il faudra que je demande à Gérald pour être certain.

Le caractère n° 4 est donc probablement crucial. Si c'est un #, je devrai sauter la ligne, c'est quand même à vérifier.

Le code en 9 est décrit dans le fichier "NouveauxCodes.DOC".

Par convention, les lemmes sont écrits en majuscule, avec le "V" en guise de u majuscule (ce qui est un des usages les plus débilés qui soient¹ –désolé si vous en êtes un adepte). Les noms propres sont associés à l'indice d'homonymie N, même s'il n'y a pas d'homonyme (dans quelques rares cas, il peut y avoir des homonymes de noms propres² qui reçoivent alors l'indice O³), et les adjectifs associés ont un indice A. La première lettre de la forme associée est alors en majuscule, alors que les formes, habituellement, sont en minuscule. Pour les lemmes sans homonyme, l'indice est un blanc. Sinon, les homonymes sont numérotés par un chiffre (je ne garantis pas qu'il n'y ait pas dans le lot un lemme numéroté 2 sans que le 1 ne soit utilisé).

1 Il faut appeler un chat, un chat. Pierre de la Ramée a introduit l'idée de distinguer les lettres u/v et i/j en 1562 (d'autres avant lui hors de France). Les grands lexicographes suivent cette tradition (par exemple, Forcellini 1771 et Gaffiot 1937 sqq). À une époque plus récente, probablement sous l'influence de gourous rétrogrades, certains dictionnaires ont abandonné ce sain principe (par exemple, l'Oxford Latin qui nomme le gourou en question). Aussi tenue que soit la justification (historique ou linguistique) de cette distinction, elle devrait être maintenue dans toute entreprise à caractère scientifique pour la simple raison qu'il est facile de perdre de l'information et beaucoup plus difficile de la restaurer. À titre d'exemple, *voluit* contient plus d'information que *uoluit* qui peut aussi être *volvit* (certes, un latiniste n'hésite pas une seconde, mais une machine ne voit que ce qui est écrit).

2 Dans un ancien format (DAT ?), les personnages différents portant le même nom étaient distingués (avec un chiffre suivant le N), mais je pense que ça a été abandonné parce que, d'une part, ce n'est pas tenable (qui sait qui est PHILIPPVS N1 et PHILIPPVS N6 ?) et, d'autre part, ça n'est pas justifié en tant que lemme.

3 Pallas, adis, f. = PALLAS N et Pallas, antis, m. = PALLAS O. Les autres indices O sont ILIADES, MEGARA, OLYMPIA et PYTHIA (pas sûr qu'il n'y ait pas de mélange pour les deux ILIADES –Iliades, ae, m. et Iliades, um, f. pl.– comme pour les deux PALLAS).

Le Tagueur avec GUI

Une première version du tagueur a été faite en 2016 (?). Elle s'appuyait principalement sur les formes issues du dépouillement des textes du LASLA et ne faisait appel à Collatinus (via un port interne, Collatinus tournant en arrière plan avec son serveur actif) que si la forme n'était pas reconnue. Margherita en a essuyé les plâtres et a constaté la faiblesse de cette approche. Le problème venait des formes identiques (par exemple, aux datif et ablatif pluriels) qui étaient suffisamment rares pour que les textes du LASLA ne contiennent qu'une seule des deux analyses et pas l'autre. L'approche adoptée initialement conduisait à une analyse unique, mais pouvant être fausse. L'erreur se propageant ensuite aux autres analyses dans la phrase par le principe même du tagueur probabiliste.

Une deuxième version du tagueur a été faite en 2017 (?) qui s'appuyait sur une double analyse des formes, à la fois avec le lexique issu du LASLA et avec Collatinus. L'appel à Collatinus étant systématique, il ne pouvait plus se faire via le serveur : les modules nécessaires ont été copiés et compilés avec le tagueur. Un bug a été découvert quand un deuxième utilisateur est apparu (en 2019) et a été corrigé dans la foulée (d'où la version 2.1).

Entre temps, Collatinus a beaucoup évolué avec, en particulier, la gestion des graphies médiévales. Il serait donc intéressant de mettre à jour, dans les sources du tagueur, celles issues de Collatinus. Le problème étant que je ne sais pas combien de changements ont été faits et j'ignore si la compatibilité est assurée.

Le programme a été conçu pour prendre un texte dans un format standard et lisible, sans traitement préalable. Quelques précautions sont toutefois nécessaires avec les sauts de ligne qui peuvent être groupés pour revêtir une signification particulière (voir plus bas). Le programme procède en deux étapes distinctes : une première que je nommerais “lemmatisation”⁴ et une seconde qui est la désambiguïsation pour aboutir à ce qui est usuellement désigné sous le terme de lemmatisation (ici, idéalement, le bon lemme avec la bonne analyse).

Comme je l'ai dit plus haut, la version actuelle du tagueur procède à une double “lemmatisation” : la première lemmatisation consiste simplement à chercher la forme parmi toutes celles rencontrées dans les textes du LASLA. On a ainsi directement les lemmes et analyses au bon format. La “lemmatisation” par Collatinus est basée sur les règles de flexion, mais inversées : on découpe la forme de toutes les façons possibles et on regarde si les deux morceaux correspondent à un “radical” et à une “désinence” compatibles⁵. Collatinus connaît plus de trois fois plus de lemmes que le LASLA n'en a répertorié et sa connaissance théorique de la flexion lui permet de reconnaître des formes qui n'ont pas été rencontrées dans les textes du LASLA. Les analyses produites par Collatinus doivent être converties au format du LASLA, tout comme les lemmes communs. Un tableau de correspondance entre les lemmes a été établi de façon automatique mais, à ma connaissance, n'a pas été vérifié systématiquement. Il faut évidemment identifier et éliminer les doublons issus de la double lemmatisation.

Une fois la lemmatisation (au sens large) effectuée, un tagueur probabiliste, fonctionnant sur le même principe que TreeTagger, va choisir la meilleure solution pour chaque mot. Son choix est limité aux résultats de la lemmatisation (ce qui peut poser quelques problèmes en cas, par exemple, d'un adjectif substantivé). Les différences se situent surtout au niveau du jeu d'étiquettes (tagset) : ici, les étiquettes sont issues des codes en 9 du LASLA et permettent, en quelque sorte, d'y revenir⁶. Une autre différence importante est que ce tagueur ne peut pas être entraîné sur un autre corpus : les statistiques nécessaires à son fonctionnement ont été faites sur les textes du LASLA par

4 Il faut noter l'absence de mot pour désigner l'opération qui consiste à décortiquer une forme pour donner tous les lemmes et toutes les analyses qui lui sont associés (toujours sous la forme de paires lemme + analyse, puisque des lemmes différents peuvent conduire à une même forme –par exemple *oris*– mais avec des analyses différentes).

5 Je mets des guillemets autour des termes radical et désinence pour ne pas avoir de comptes à rendre aux linguistes. Les désinences sont associées à une analyse et à un paradigme et les radicaux le sont à un lemme et un paradigme. En éliminant, le paradigme commun (condition de compatibilité) on associe donc la forme, comme paires radical + désinence, à une collection de paires lemme + analyse.

6 L'intégralité des codes en 9 formerait un jeu trop vaste (plus de 3000 étiquettes). La restriction que j'ai retenue compte probablement déjà trop d'étiquettes.

un autre programme. Les solutions proposées par l'ordinateur sont ensuite soumises à la lecture critique par un philologue qui va corriger le fichier APN. Bien qu'il soit possible de modifier un fichier APN dans un éditeur de textes, il a paru plus sain de donner au philologue le choix parmi les lemmatisations proposées (avec la possibilité d'en ajouter des nouvelles).

Dans l'utilisation du programme, le philologue intervient à quatre niveaux. D'abord, dans le choix du fichier à traiter. Ensuite, il doit indiquer au programme le code de l'œuvre et la façon de gérer les sauts de ligne pour construire la référence présente sur chaque ligne du fichier APN. De plus, nous avons prévu le cas où une même œuvre serait découpée en plusieurs fichiers. Les détails sur ces deux points sont donnés dans le paragraphe suivant. Lors de la lemmatisation, le philologue va devoir intervenir pour donner la lemmatisation et l'analyse des formes qui ne sont pas reconnues par le programme. Enfin, le philologue corrige le fichier APN puisqu'un programme peut commettre des erreurs et que l'on veut un résultat parfait⁷.

Sauts de ligne et référencement

Sur chaque ligne d'un fichier APN figure une référence : 11 caractères sont réservés pour cela et ce champ peut contenir jusqu'à trois nombres séparés par une virgule. Ils servent à repérer où on en est dans le texte et leur nombre dépend de la structure du texte. Si on a juste un poème, le numéro du vers suffit. Si on a un recueil de poèmes, outre le numéro du vers, on a besoin de numérotter les poèmes. Pour une œuvre poétique plus conséquente, on aura une division en livres, poèmes et vers. Pour une œuvre en prose, l'unité de base est le paragraphe et peuvent s'ajouter les chapitres et les livres. Dans certains textes en prose, les lignes sont marquées par un saut de ligne et les paragraphes par un double saut de ligne (i.e. une ligne blanche entre deux paragraphes). La plupart du temps dans un tel cas, la ligne ne m'intéresse pas : je dois ignorer le saut de ligne simple et tenir compte des doubles sauts de ligne (entre les paragraphes). Si je veux marquer des chapitres, j'emploierai des triples sauts et des quadruples sauts de ligne entre les livres. Dans certains poèmes, on peut avoir des strophes séparées par une ligne blanche (double saut de ligne), mais on ne souhaite pas garder cette structure dans la référence : le double saut de ligne doit être ignoré (considéré pour la référence comme un simple saut) et la séparation entre les poèmes devra compter deux lignes blanches (triple saut).

Dans le cas où l'œuvre est considérable, on peut être tenté de la découper en plusieurs fichiers, par exemple un fichier pour chaque chapitre, que l'on traitera de façon séquentielle. Mais on voudrait aussi qu'à la fin les fichiers APN obtenus puissent être regroupés en un seul fichier. Il faut pouvoir indiquer que le deuxième chapitre aura une référence commençant par 2 (et 3 pour le suivant etc.). Ce n'est pourtant pas suffisant, car les phrases sont également numérotées (les caractères 5 à 8 de chaque ligne). Il est donc prévu que l'on puisse indiquer un décalage dans le comptage des phrases. Quand on a traité le premier chapitre, on va voir combien de phrases il contient (disons 112) et on demande à ce que la première phrase du 2^{ème} chapitre soit numérotée continûment (c'est à dire 113).

Pour traiter l'ensemble de ces cas théoriques, j'ai introduit la fenêtre de sélection ci-contre (ici, le contenu est celui que je devrais donner pour mon 2^{ème} chapitre de l'œuvre dont le code est Bla, commençant avec la phrase n° 113). Je suppose ici que les lignes ne sont pas marquées et que les sauts de ligne simples séparent les paragraphes que je numérote en commençant à 1. J'ai supposé aussi que mon texte ne contient pas de saut quadruple et que le numéro du

/Users/Philippe/Documents/LASLA/essai2.txt

LASLA

Pour obtenir le format APN, vous devez indiquer :

- la référence de l'œuvre en trois caractères éventuellement suivie de '&' et du numéro de la 1ère phrase
- les champs qui doivent repérer le mot dans le texte «Po,V» ou «L,Ch,Par». Peuvent être décalés. Par défaut, le programme compte les sauts de ligne.

Bla&113

☒ Saut de ligne simple 1

☐ Saut de ligne double 1

☐ Saut de ligne triple 1

☒ Saut de ligne quadruple 2

Appliquer

⁷ En réalité, on ne sait pas à quel taux d'erreur l'utilisation ultérieure sera sensible. Ne sachant pas fixer de seuil tolérable, on est condamné à opter pour la seule valeur singulière : 0.

chapitre restera 2 pour l'ensemble du fichier. Si mon texte contient des sauts doubles ou triples, ils seront considérés comme équivalents à un saut simple. Si mes lignes étaient marquées et que les paragraphes étaient séparés par des doubles sauts de ligne, c'est la deuxième case qui serait cochée (et pas la première).

Tagueur intégré à HyperBase

L'idée d'intégrer une nouvelle version du tagueur à HyperBase a été formulée par Dominique en septembre 2021. Il faudrait donc une version autonome, ne requérant pas l'intervention d'un philologue. Or, on a vu ci-dessus qu'il intervenait à quatre étapes. La dernière, la post-correction, peut être abandonnée sans état d'âme : le résultat n'est pas parfait, mais on va espérer qu'il sera suffisamment bon pour ne pas fausser les études que l'on veut faire derrière.

Un coup de main était demandé au philologue pour analyser les formes que le programme ne reconnaissait pas. Une solution sauvage serait de lui attribuer le lemme "VNKNOWN" (pour rester en majuscule) et toutes les étiquettes possibles. Depuis quelques mois, ma version de Collatinus (sur mon ordinateur exclusivement) essaie d'identifier les formes inconnues en s'appuyant sur la seule désinence. Je ne sais pas encore quelle en est l'efficacité. De plus, comme pour la gestion des graphies médiévales, la mise à jour du moteur de Collatinus peut passer comme une lettre à la poste ou devenir un véritable casse-tête...

Le choix du fichier est évident : c'est celui que l'utilisateur vient d'up-loader. Le système hôte le connaît donc et peut le passer à la ligne de commande ou au port interne (aujourd'hui, je penche en faveur de cette option avec un démon qui tourne en permanence). Le nom du (ou des) fichier(s) de sortie se déduit automatiquement de celui du fichier d'entrée. Restent donc le choix du code de l'œuvre et la gestion des sauts de ligne, points détaillés un peu plus haut. Le plus simple est de les passer comme paramètres optionnels en utilisant une syntaxe dans le style Unix. Je propose d'utiliser, d'une part, -N pour introduire le code de l'œuvre et, d'autre part, -f et -F pour définir le format du référencement (la minuscule pour dire que l'on considère le saut de ligne simple et la majuscule pour l'ignorer). Ce qui suit le -N est le code de l'œuvre en trois caractères (il en faut au moins un, sinon on reprend le code par défaut, "Zxz" par exemple ; s'il y en a moins de trois, on complète avec des blancs ; s'il y en a plus de trois, on tronque à trois). Éventuellement, pour traiter le cas d'une œuvre en plusieurs fichiers, j'ajoute après le code un "_" (tiret du 8, caractère qu'Unix aime bien) suivi du numéro de la première phrase : dans le cas de l'exemple donné plus haut, on aurait -NBla_113. Ce qui suit le -f (ou le -F) donne les numéros du premier paragraphe, chapitre et livre (si nécessaire) séparés par des __. Comme il est absurde de numéroter 0 un premier paragraphe (dans la vie courante), on peut utiliser ce chiffre pour dire que l'on souhaite ignorer les sauts doubles ou triples en fonction de la position. Pour reproduire l'exemple ci-dessus, on utilisera le code -fl_0_0_2.

Si j'ajoute une option "graphies médiévales", il faudra mettre une option -M ? Est-ce que l'on prévoit des extensions de lexique ciblées ?...

Une question qui se pose est de savoir si on veut pouvoir proposer, comme le fait la version avec GUI, de traiter des textes saucissonnés en plusieurs fichiers. Ça n'est pas compliqué à programmer pour moi. C'est un peu plus lourd au niveau de l'interface web et donc pour l'utilisateur : cela double le nombre de cases à considérer.